

→ strings → holds the data of only 'i' type.
↙
iterable. string with multiple words.

→ print words in python 2, 3

without print only in python 3

→ len('Hello world')

↓
'11'

→ string indexing ✓

→ string slicing

↓
: → to perform slicing.
len(word).
S [: : 2]
↓ ↓ ↓
start end step size.

S [: :] , S (:) → ~~both~~ everything.

S [- 3] → accessing from end.

S [: - 1] , S [: : - 1] → reverse the string.
↓ ↓
what is the difference.

Start at ~~begin~~ beginning
and fetch ~~from~~ till last

No slicing in interviews

String properties:-

↓
immutable object (can't change the value).

↓
can't modify data, not possible for
item assignment.

concatination is possible.

(is a new reference created)??

$S = S + 'x'$

↓
reassigning the string.

$z = \text{letter}$

$\text{letter} * 3 = z z z$

→ Built in string:-

$s.$ upper(), $s.$ lower(), $s.$ split()
↓
white space.

$s.$ split() → any delimiter.

↓
delimiter is not included
in the output.

S. join (l → d)

• format (l, f"

→ location and counting.

S. count ('o'), S.find ('o')

→ expand tabs

\t → replaced with tab space.

→ is check methods.

S. isalnum()

↳ checks for alpha ^{or} numeric.

S. islower()

S. istitle()

S. isupper()

↳ checks for first letter is upper.

S. ends with ('o')

↳ check for end.

→ reg exp.

S. partition (l) → similar to split

but includes the delimiters.

S. split(.)

S. partition(.).

→ Lists

↳ different types of data types.

↳ mutable.

↳ editable

→ indexing slicing → feature.

→ append, pop(lifo)

→ Nested list

↓
matrix

2-dimensional array

matrix [x][y].

matrix [0][0][1].

→ list comprehension

first_col = [i[0] for i in matrix]

↓ ↓
first each list in matrix
index
of each list.

→ practice list comprehensions

→ Advanced list

l.count()

l.extend() (vs) append() → at the end.
↓
combines the list. ↓
list & list

insert()

↓
at specific

position.

remove()

↓
at specific

position

pop()

↓
at last

reverse()

sort()

↓
asc, desc.

→ Tuples → permanent storage.
→ Convert to list and edit
↓
immutable

→ sets (eliminate the duplicates).
↓
discard, remove → only if exists.
↓
pop remove

→ Clear, complete removal.

→ dictionary → hash tables.

↓
key value to pairs.

↓
handles duplication

↓
only takes the last assignments
ignores the initial assignments

↓
nested dictionaries.

`dict[key] = value` → accessing. → important.

↓
dictionary comprehension

d. keys
d. values
d. items

→ Shallow ~~copy~~ copy

import copy.

original.

↓
shallow = copy.copy(original)

copy.deepcopy(original).